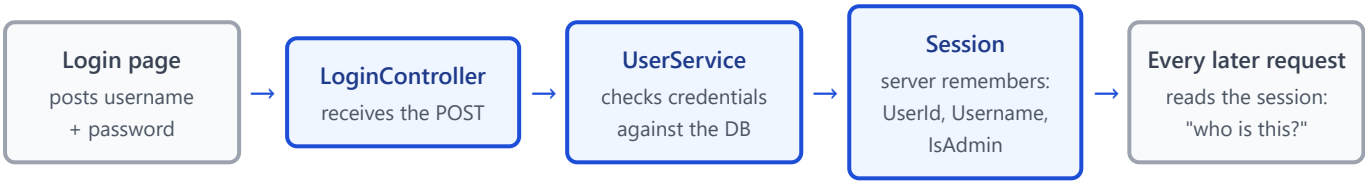
QUOTESPROJECT — BUILD GUIDE • PART 1 OF 2

Login & Sessions — Step-by-Step Setup

Build the login backend and store the logged-in user in session state — the exact flow, no helper layers. Part 2 adds the IsAdmin authorization on top.

0. What you're building

At the end of this guide, logging in works and *sticks*: the server remembers who each visitor is between requests. The whole design is five moves:



How session state works, in one paragraph: when a visitor first gets session data, ASP.NET Core creates a random session id, sends it to the browser in a cookie, and keeps a small data bag for that id *on the server*. On every later request the browser automatically sends the cookie back, the middleware looks up the id, and `HttpContext.Session` is that visitor's bag. The browser only ever holds the meaningless id — the actual data (who is logged in) never leaves the server.

Prerequisites: the `Users` table exists (run `Scripts/CreateUsersTable.sql` — it seeds `admin`/1234 and `user`/1234, with the passwords stored as salted hashes, not text; the password you *type* is still 1234), and the login view exists at `Views/Login/Index.cshtml`. The view defines the contract your controller must satisfy: **GET** `/Login` returns the page, and **POST** `/Login` receives JSON `{ "username": "...", "password": "..." }`, answering 2xx on success (the page then redirects to `/Quote`) or 401 with a message on failure.

Build order below matters: each step compiles against the one before it. Models first, then data access, then the service, then wiring, then the controller.

Step 1 — Create the User model

`Models/User.cs  (new file)`

```
namespace QuotesProject.Models
{
    public class User
    {
        public int UserId { get; set; }
        public string Username { get; set; }
        public string Password { get; set; }
        public int IsAdmin { get; set; }   // IsAdmin column: 1 = admin, 0 = regular user
    }
}
```

What it does & why: mirrors one row of the `Users` table, exactly like `Quote` mirrors the Quote table. The database column stores `1` (admin) or `0` (regular user), and the model keeps it as a plain `int` — no bool, no true/false translation, the 0/1 flows through unchanged and is read in Step 4. Nothing else in the app should ever touch raw table data; it flows through this class.

Step 2 — Create the LoginRequest model

`Models/LoginRequest.cs  (new file)`

```
namespace QuotesProject.Models
{
    public class LoginRequest
    {
        public string Username { get; set; }
        public string Password { get; set; }
    }
}
```

Why a second model instead of reusing `User`: this class is what the browser is *allowed to send*. If the POST bound straight to `User`, a crafted request could include `"isAdmin": true` in the JSON and the model binder would happily set it (called *over-posting*). With `LoginRequest`, the binder physically has nowhere to put extra fields. Rule worth keeping: input models contain only what the client may supply.

Step 3 — Add the query to DatabaseEngine

`Data/DatabaseEngine.cs  (add this method)`

```
//Users

public static DataTable GetUserByUsername(string username)
{
    string query = @"
                    SELECT
                        UserId,
                        Username,
                        Password,
                        IsAdmin
                    FROM Users
                    WHERE Username = @Username";

    using var command = new SqlCommand(query);

    command.Parameters.AddWithValue("@Username", username);

    return ExecuteDataTable(command);
}
```

What it does & why: fetches the row for one username — and nothing more. No password checking here: the engine's job is SQL, decisions belong to the service. Two deliberate details: the `@Username` parameter

means user input is never concatenated into SQL (this is what blocks SQL injection — a username of `' OR 1=1 --` is just a weird string, not executable SQL), and we fetch by username alone rather than `WHERE Username = @u AND Password = @p` so the *comparison* happens in C# where you control it — which is essential here, because the stored value is a salted hash and only C# knows how to verify against it (Step 4).

Step 4 — Create the service (interface + implementation)

`Interfaces/IUserService.cs  (new file)`

```
using QuotesProject.Models;

namespace QuotesProject.Interfaces
{
    public interface IUserService
    {
        User ValidateUser(string username, string password);
    }
}
```

`Services/UserService.cs  (new file)`

```
using Microsoft.AspNetCore.Identity;   // PasswordHasher lives here (ships with ASP.NET Core)
using QuotesProject.Data;
using QuotesProject.Interfaces;
using QuotesProject.Models;
using System.Data;

namespace QuotesProject.Services
{
    public class UserService : IUserService
    {
        public User ValidateUser(string username, string password)
        {
            if (string.IsNullOrWhiteSpace(username))
                throw new ArgumentException("Username is required.");

            if (string.IsNullOrWhiteSpace(password))
                throw new ArgumentException("Password is required.");

            DataTable userTable = DatabaseEngine.GetUserByUsername(username);

            if (userTable.Rows.Count == 0)
                throw new UnauthorizedAccessException("Invalid username or password.");

            DataRow row = userTable.Rows[0];

            User user = new User
            {
                UserId   = Convert.ToInt32(row["UserId"]),
                Username = row["Username"].ToString(),
                Password = row["Password"].ToString(),        // the stored salted HASH — never
plain text
                IsAdmin  = Convert.ToInt32(row["IsAdmin"])   // stored as 0/1, read straight into
the int
            };

            // Verify the typed password against the stored salted hash. The hash blob
            // contains its own salt: VerifyHashedPassword unpacks it, re-runs the same
            // hashing on what was typed, and compares the results. The typed password
            // is never compared as text, and the stored value cannot be reversed.
            var hasher = new PasswordHasher<User>();
            var result = hasher.VerifyHashedPassword(user, user.Password, password);

            if (result == PasswordVerificationResult.Failed)
                throw new UnauthorizedAccessException("Invalid username or password.");

            return user;
        }
    }
}
```

What it does & why: this method IS authentication — the single place that decides "are these credentials real?". Same layering as `QuoteService`: controller orchestrates, service decides, engine fetches. The interface

exists so the controller depends on a contract, not a concrete class (same reason as `IQuoteService`).

How the hash verification works, step by step: the Password column never holds a password — it holds a one-way *salted hash* (an `AQAAAA...` blob, created when the user was created; the full hashing explainer is in Part 3, Section 0.5). Login therefore cannot ask "is the stored value equal to what was typed?" — it asks "does what was typed *hash to* the stored value?":

- `VerifyHashedPassword(user, storedHash, typedPassword)` unpacks the blob — algorithm, iteration count, and crucially the **salt** are all stored inside it (no separate salt column needed).
- It re-runs the identical PBKDF2 computation on the *typed* password using that same salt, and compares the outcome to the stored one. Match → the user knew the password; mismatch → `Failed`.
- The comparison result can also be `SuccessRehashNeeded` — meaning "correct password, but the hash was made with older settings". Checking `== Failed` treats that as a successful login, which is right; upgrading the stored hash on such logins is an optional refinement for later.
- Note what this buys you: even with full database access, nobody — including you — can read anyone's password. A stolen backup leaks blobs that each have to be brute-forced individually, slowly.

Why the SAME message for unknown user and wrong password

Both failures throw `UnauthorizedAccessException("Invalid username or password.")`. If the messages differed ("no such user" vs "wrong password"), the login form would let an attacker discover which usernames exist by trying names and reading the error. One indistinguishable message closes that leak.

Step 5 — Register the service in Program.cs

`Program.cs  (add one line to the existing service registrations)`

```
// Register services
builder.Services.AddScoped<IQuoteService, QuoteService>();
builder.Services.AddScoped<IQuoteLineService, QuoteLineService>();
builder.Services.AddScoped<IUserService, UserService>();          // ← add this
```

What it does & why: tells dependency injection "when a constructor asks for `IUserService`, hand it a `UserService`". `AddScoped` = one instance per HTTP request, matching your other services. Without this line, creating `LoginController` would fail at runtime with "Unable to resolve service for type IUserService".

Step 6 — Enable session state in Program.cs

Two additions: register the session services, then put the middleware in the pipeline.

`Program.cs (services section — add below the service registrations)`

```
builder.Services.AddDistributedMemoryCache();      // the store where session data lives (in-memory)

builder.Services.AddSession(options =>
{
    options.Cookie.Name = ".QuotesProject.Session";             // recognisable in dev-tools
    options.Cookie.HttpOnly = true;                              // no JS access (default anyway)
    options.Cookie.IsEssential = true;                           // works even before cookie-consent is given
    options.Cookie.SecurePolicy = CookieSecurePolicy.Always;     // HTTPS only
    options.Cookie.SameSite = SameSiteMode.Lax;                  // CSRF mitigation (Lax is default)
    options.IdleTimeout = TimeSpan.FromMinutes(30);              // 30 min of inactivity → session erased
});
```

`Program.cs (pipeline section — add one line after UseRouting)`

```
app.UseHttpsRedirection();
app.UseRouting();

app.UseSession();          // ← add this: attaches the session bag to each request

app.UseAuthorization();
```

What each piece does:

- **`AddDistributedMemoryCache()`** — sessions need somewhere to keep the data; this says "in this app's memory". (In a multi-server deployment you'd swap only this line for Redis or SQL Server — the rest of your code wouldn't change.)
- **`AddSession(options)`** — registers the session feature and configures its id cookie. `HttpOnly` keeps the id out of JavaScript's reach (so an injected script can't steal a logged-in session). `IdleTimeout` is the security valve: an abandoned machine logs itself out after 30 quiet minutes; every request from the user resets the clock.
- **`SecurePolicy = Always`** — the browser will only ever send the session cookie over HTTPS. Why it matters: whoever holds the session id *is* that user as far as the server can tell (session hijacking), so one accidental plain-HTTP request leaking the id on the network is one too many. Pairs with your existing `UseHttpsRedirection()`. Practical note: with this on, always test via the `https://` URL — over plain `http://` the cookie is never set and login won't stick.
- **`SameSite = Lax`** — the browser refuses to attach the session cookie when *another site* triggers a POST to yours, so a malicious page can't fire state-changing requests (delete a quote, log you out) that ride on your users' logged-in sessions — that attack is CSRF. `Lax` is the modern browser default, and `HttpOnly =`

`true` is also a default — both are written out explicitly so the protection is visible and deliberate rather than accidental.

- **`app.UseSession()`** — the middleware that, on every request, reads the id cookie and makes that visitor's bag available as `HttpContext.Session`. **Order rule:** it must appear *before* anything that reads the session — and since your controllers will read it, "after `UseRouting()`, before the endpoints run" is the correct spot.

Two things that surprise people about the memory store

(1) Restarting the app erases every session — everyone is logged out. Fine in development; in production you'd use a persistent store. (2) The 30-minute idle timeout is *independent* of the browser: the cookie may still exist while the server-side bag has already been erased. Your checks must therefore always ask the session, never assume the cookie means anything by itself.

Step 7 — Create the LoginController

`Controllers/LoginController.cs  (new file)`

```
using Microsoft.AspNetCore.Mvc;
using QuotesProject.Interfaces;
using QuotesProject.Models;

namespace QuotesProject.Controllers
{
    [ApiController]
    [Route("[controller]")]
    public class LoginController : Controller
    {
        private readonly IUserService _userService;

        public LoginController(IUserService userService)
        {
            _userService = userService;
        }

        [HttpGet]
        public IActionResult Index()
        {
            return View();                      // serves Views/Login/Index.cshtml
        }

        [HttpPost]
        public IActionResult Login(LoginRequest login)
        {
            if (!ModelState.IsValid)
                return BadRequest(ModelState);

            try
            {
                var user = _userService.ValidateUser(login.Username, login.Password);

                // The login "sticks" HERE. Three values into this visitor's session:
                HttpContext.Session.SetInt32("UserId", user.UserId);
                HttpContext.Session.SetString("Username", user.Username);
                HttpContext.Session.SetInt32("IsAdmin", user.IsAdmin);

                return Ok(new { user.UserId, user.Username, user.IsAdmin });
            }
            catch (ArgumentException ex)
            {
                return BadRequest(ex.Message);
            }
            catch (UnauthorizedAccessException ex)
            {
                return Unauthorized(ex.Message);
            }
            catch (Exception ex)
            {
                return StatusCode(500, "Error logging in: " + ex.Message);
            }
```

```
        }

        [HttpPost("Logout")]
        public IActionResult Logout()
        {
            HttpContext.Session.Clear();             // erase this visitor's bag → logged out
            return Ok();
        }
    }
}
```

What it does & why, piece by piece:

- **The three `Session.Set...` lines are the entire login mechanism.** Before them, validation succeeded but the server would forget it by the next request. After them, every future request from this browser can ask "who is this?" and get an answer. `SetInt32` / `SetString` are the framework's built-in accessors (session stores raw bytes; these are the standard typed wrappers — not custom helpers).
- **`IsAdmin` goes into the session unchanged** — it's already an `int` (1 = admin, 0 = regular user), so `SetInt32` takes it directly, with no `? 1 : 0` conversion needed. It mirrors the database column exactly. Part 2 reads it back with `GetInt32("IsAdmin") == 1`.
- **Wrong credentials never touch the session** — the exception jumps straight to the `catch`, so a failed login leaves the visitor exactly as anonymous as before.
- **Logout is a POST, not a GET** — browsers prefetch GET links and third-party pages can trigger GETs; either would randomly log your users out. State-changing actions are POSTs.
- `[ApiController]` + `[Route("[controller]")]` — same pattern as your other controllers: JSON body binds to `LoginRequest` automatically, routes are `/Login` and `/Login/Logout`.

Step 8 — Reading the session (anywhere)

No new files — this is the syntax you'll use from now on. In a **controller**:

```
int? userId  = HttpContext.Session.GetInt32("UserId");     // null = not logged in
string? name = HttpContext.Session.GetString("Username");
```

In a **view** (Razor exposes the same session as `Context.Session`):

```
@{
    var username = Context.Session.GetString("Username");
}
```

The one convention that matters: *"logged in" means* `GetInt32("UserId") != null` — nothing else. The getters return `null` when the key doesn't exist, which covers every anonymous case at once: never logged in, logged out, session expired, or app restarted. One definition, used everywhere, is what keeps the design coherent.

Step 9 — A first gate (temporary, to see it work)

`Controllers/QuoteController.cs  (edit the existing Index action)`

```
[HttpGet]
public IActionResult Index()
{
    if (HttpContext.Session.GetInt32("UserId") == null)   // not logged in?
        return Redirect("/Login");

    var quotes = _quoteService.GetQuotes();
    var viewModel = new QuoteViewModel { Quotes = quotes };
    return View(viewModel);
}
```

What it does & why: your first real use of the session — the quotes page now requires a login. This inline check is deliberately bare so the flow is visible: read session → null means anonymous → redirect. It guards only this one action; protecting *every* action (including the POST/PUT/DELETE endpoints that `fetch` calls) without copy-pasting this into all of them is exactly what Part 2 solves — and this temporary check gets replaced there.

Step 10 — Test it

Do this	You should see
1. Open `/Quote` without logging in	Redirected to `/Login`.
2. Login with a wrong password, then with a made-up username	401 with the *same* message both times; dev-tools (F12 → Application → Cookies) shows **no** `.QuotesProject.Session` cookie yet.
3. Login as `admin` / 1234	Redirected to `/Quote` and it loads. The `.QuotesProject.Session` cookie now exists (HttpOnly ✓, Secure ✓, SameSite=Lax ✓).
4. Close the tab, reopen `/Quote`	Still logged in — same browser, same cookie, same server-side bag.
5. POST to `/Login/Logout` (or clear the cookie), then open `/Quote`	Redirected to `/Login` — the bag is gone.
6. Login again, then restart the app, then refresh `/Quote`	Redirected to `/Login` — the in-memory store died with the app. Expected with the memory store.

What you've built (and what's next)

This is the classic **session-id login**: the browser holds a random id, the server holds the truth. Its strengths (instant logout, nothing sensitive in the browser) and costs (server memory, shared store when scaling, lost on restart) are its permanent trade-offs, not bugs. Right now, though, *everyone who logs in can do everything* — `user` can delete quotes just like `admin`. Making the two roles mean something is Part 2.